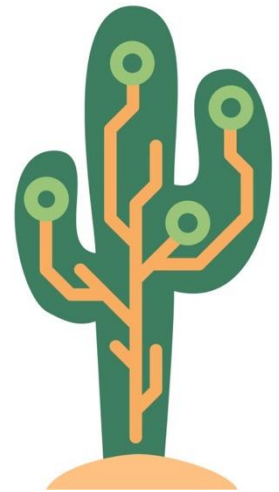


Advanced Hardware and Systems for Machine Learning

Lecture 4, 5: Deep Neural Networks

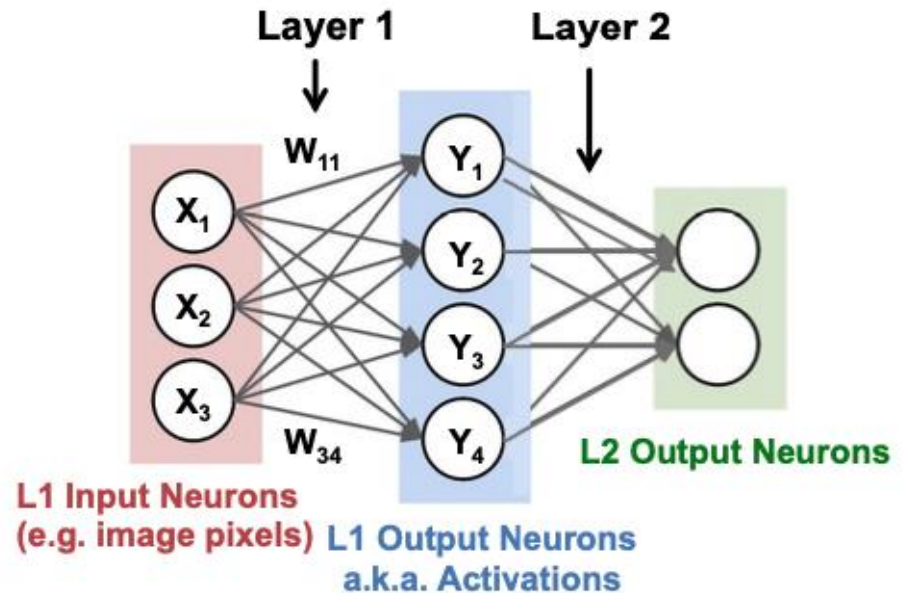
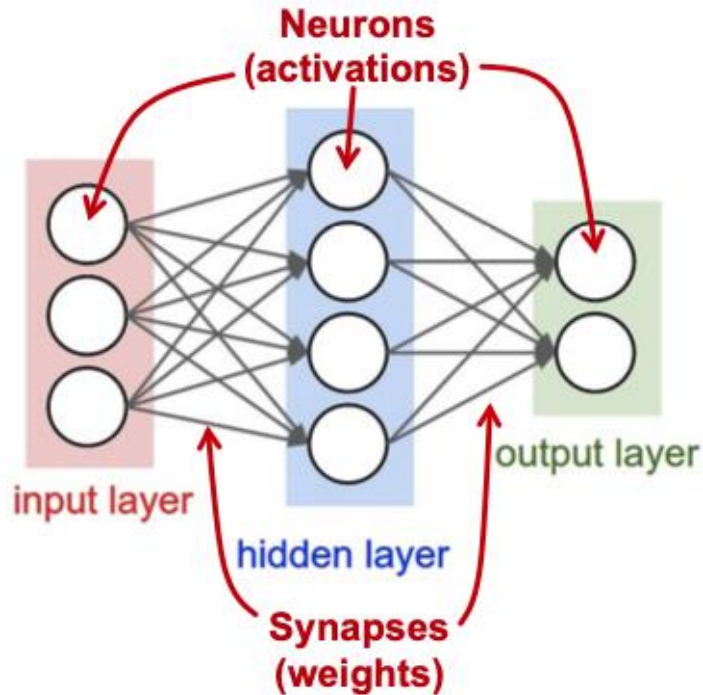


Jeff (Jun) Zhang
Assistant Professor
School of ECEE, ASU
jeffzhang@asu.edu



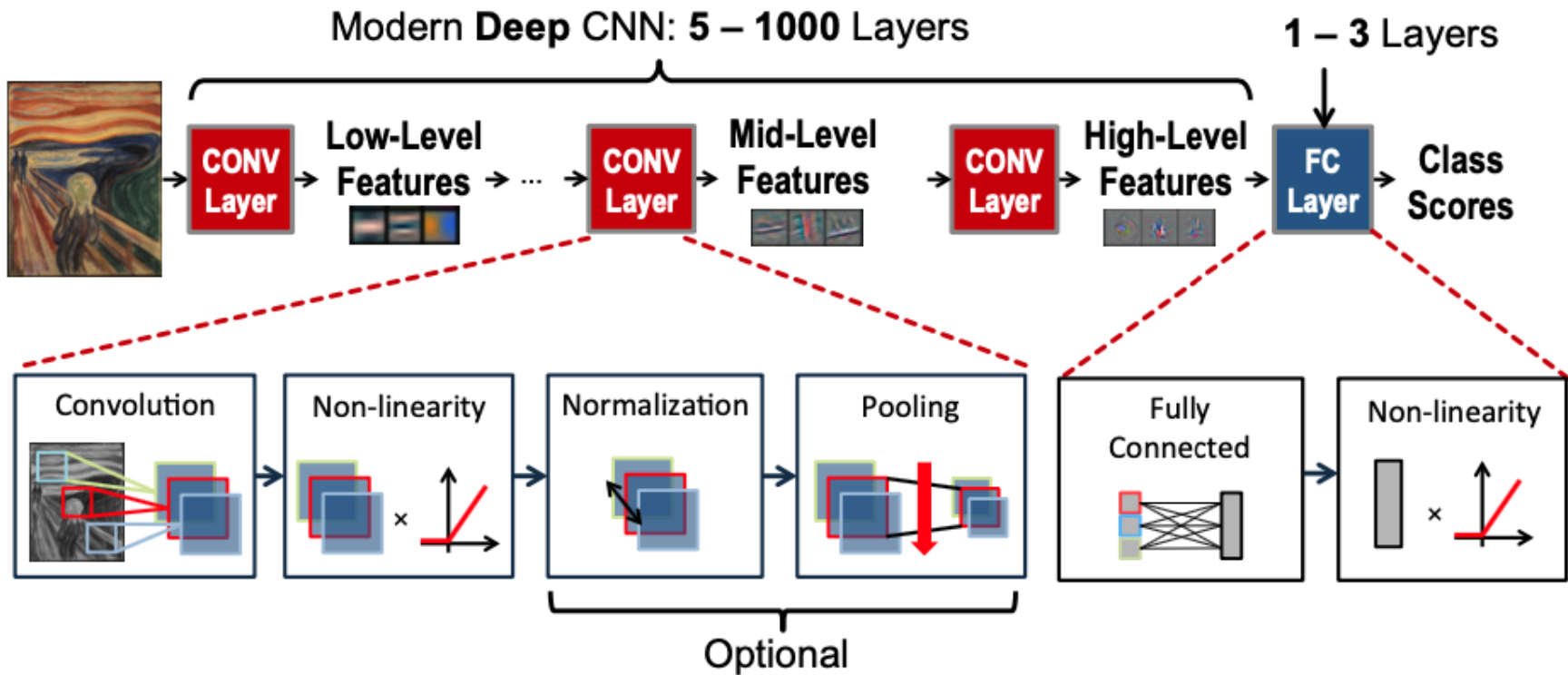
Deep Neural Networks

- **A subset of machine learning models**
 - Deep -> multiple layers
 - Linear model + nonlinear transformation



Eyeriss tutorial

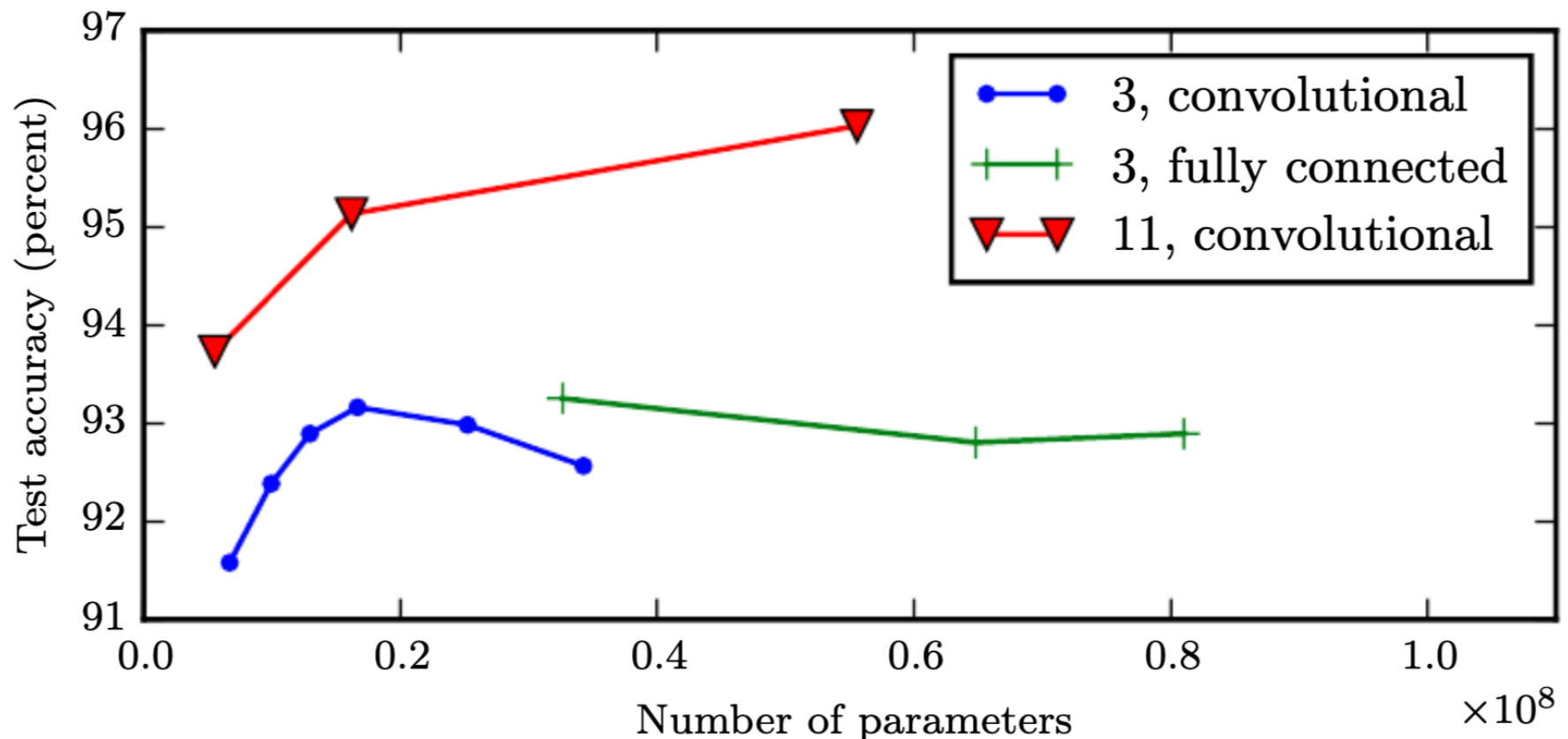
Deep Neural Networks



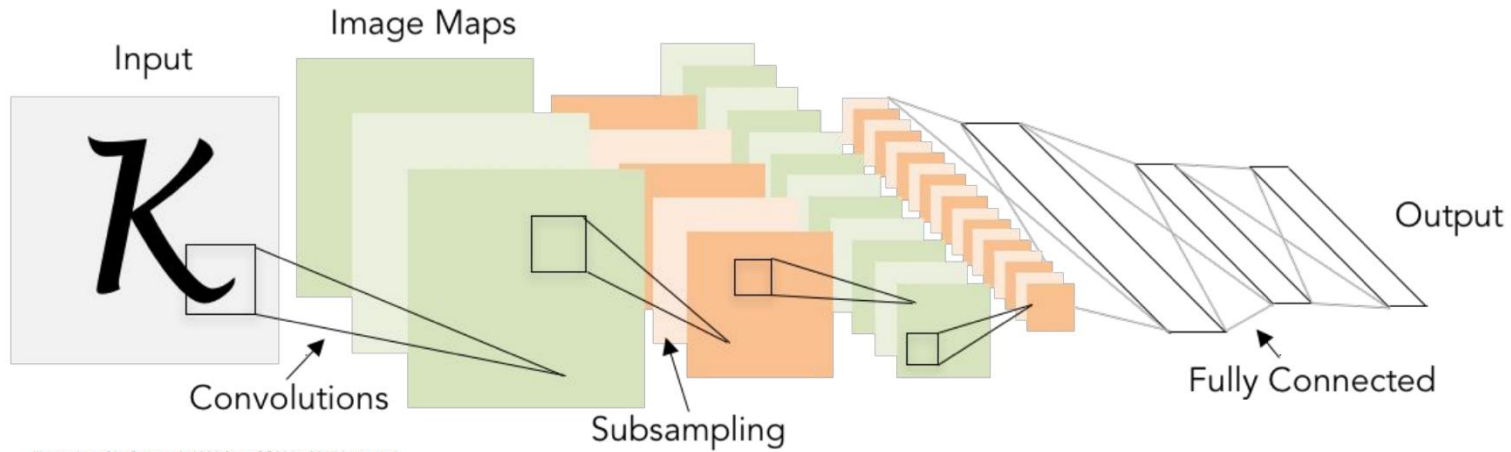
Eyeriss tutorial

DNN Parameters

■ Depth and width of DNNs



Deep Neural Networks: LeNet



0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9
0	1	2	3	4	5	6	7	8	9

AlexNet Model

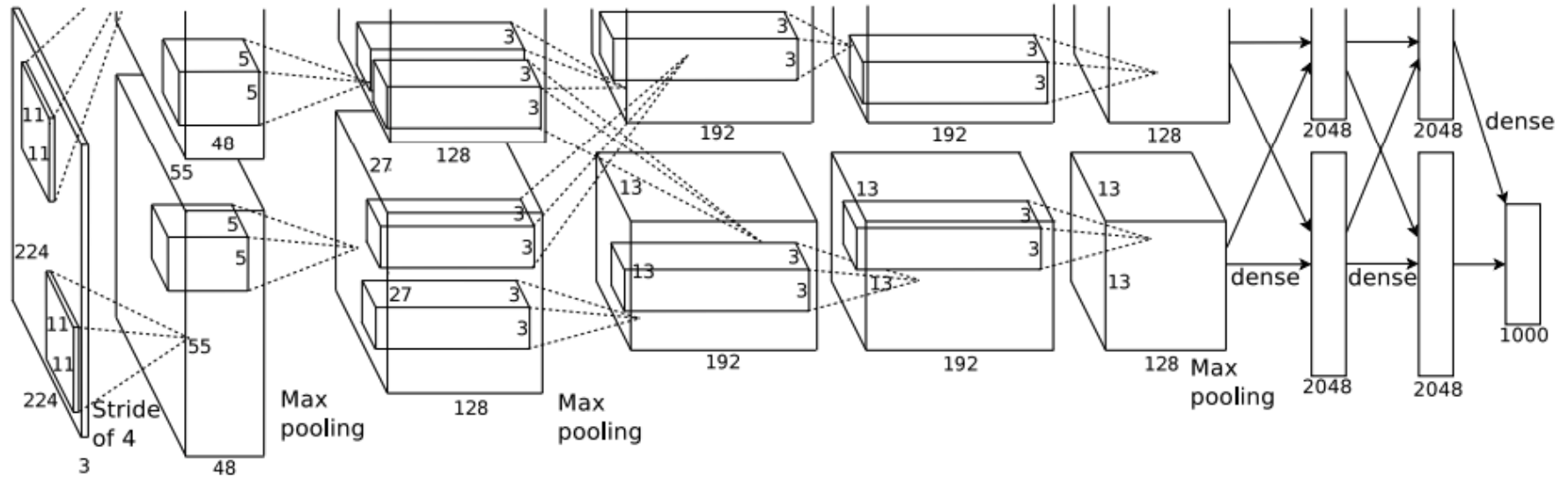
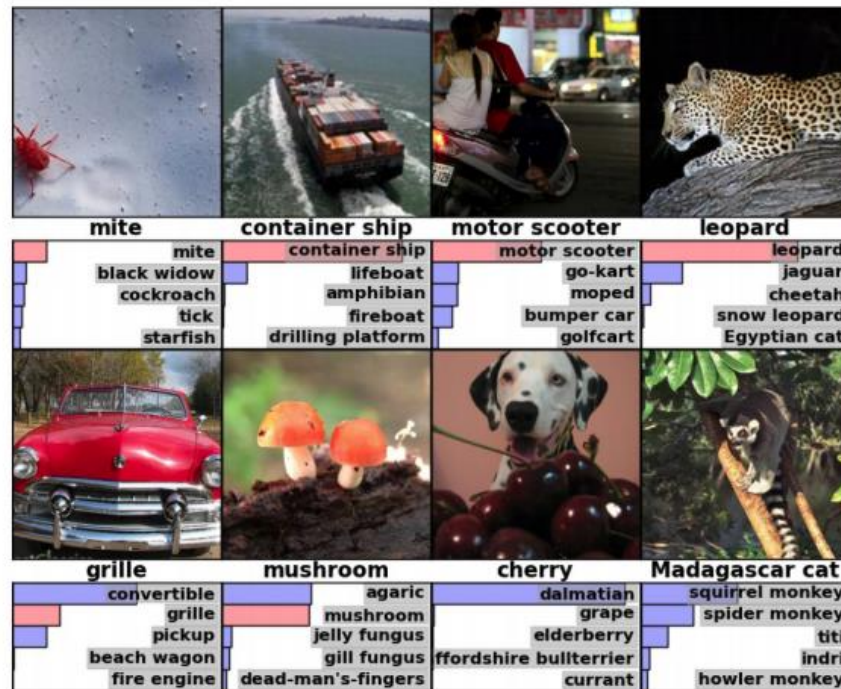


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet Dataset

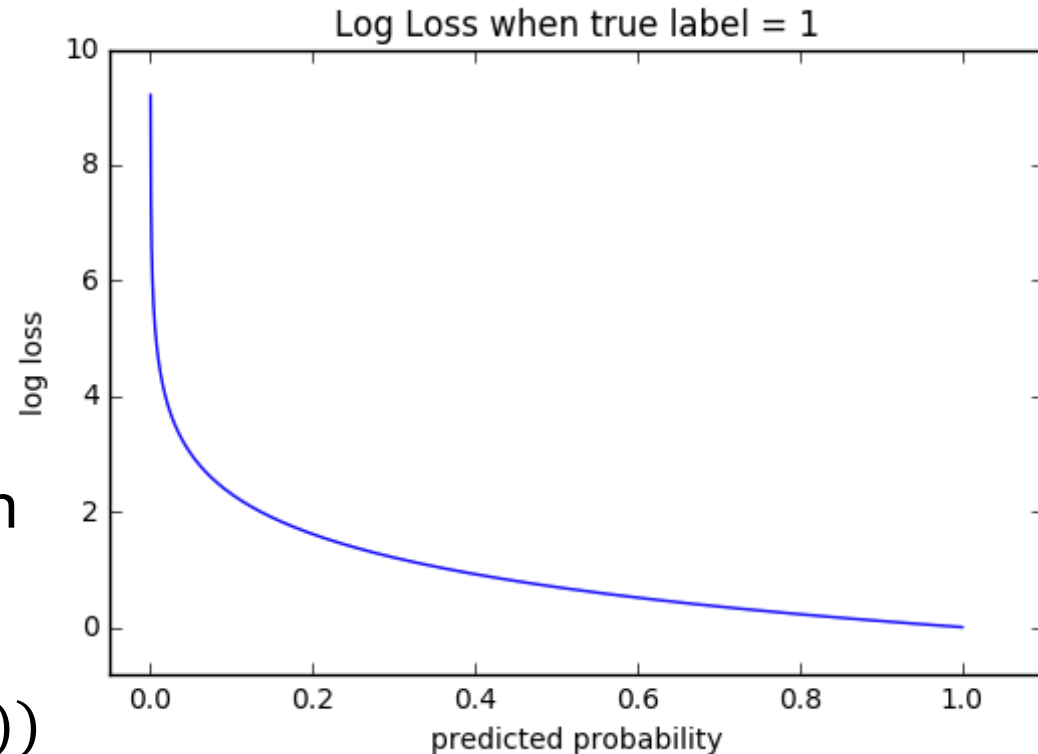
- ImageNet LSVRC-2010 with 1.2 million images
- Top-1 and top-5 error rate:
 - The fraction of images for which the correct label is not among the 1/5 labels considered most probable by the model.



[A. Krizhevsky, I. Sutskever, G.E. Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, 2012]

AlexNet Cost (loss) function

- **Minimize the cross-entropy loss function**
- Measures the performance of a classification model whose output is a probability value between 0 and 1
- $CF = -\frac{1}{N} (\sum_{i=1}^N y_i \cdot \log(\hat{y}_i))$



AlexNet Optimization Method

■ Stochastic Gradient Descent

- Batch size = 128
- Update rule:

$$\begin{aligned}
 v_{i+1} &:= \overset{\text{Momentum}}{0.9 \cdot v_i} - \overset{\text{Weight Decay}}{0.0005 \cdot \epsilon \cdot w_i} - \epsilon \cdot \left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i} \\
 w_{i+1} &:= w_i + v_{i+1}
 \end{aligned}$$

Learning rate $\uparrow$ ϵ
 Gradient of cost function $\nwarrow$ $\left\langle \frac{\partial L}{\partial w} \Big|_{w_i} \right\rangle_{D_i}$

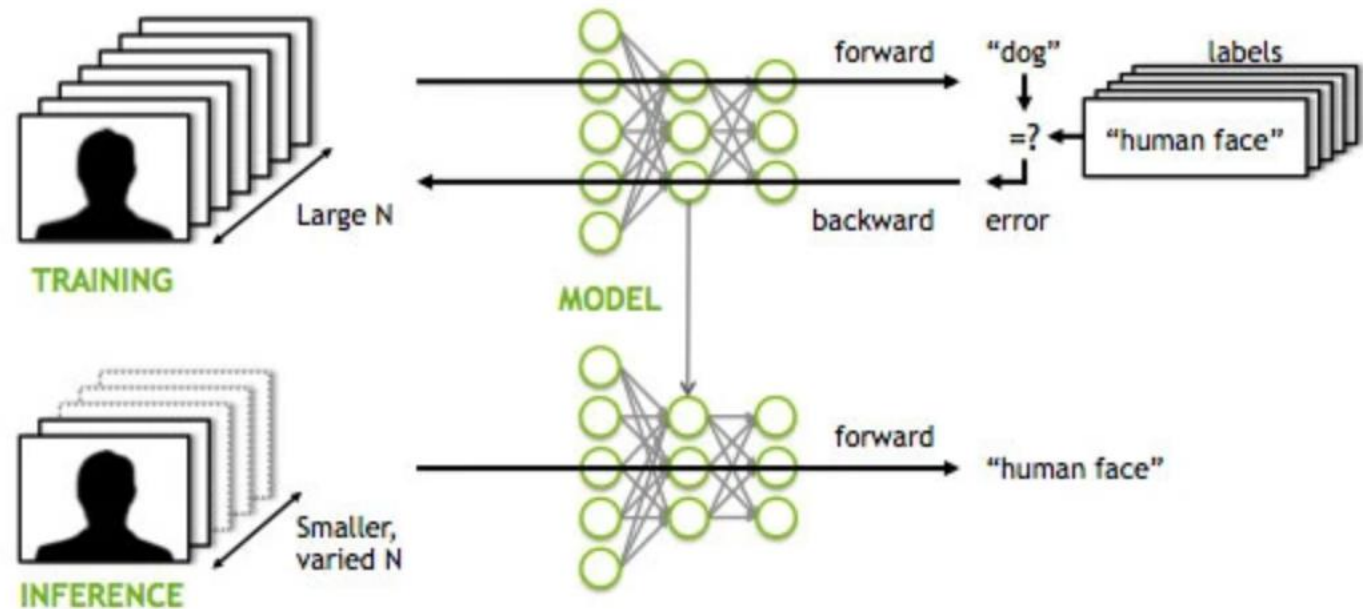
Training vs Inference

■ Training

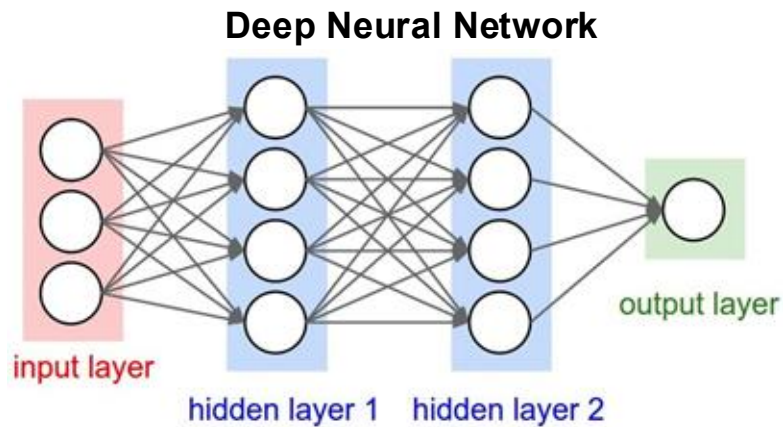
- Dataset
- Cost function
- Optimization function
- Model

■ Inference

- Dataset
- Model

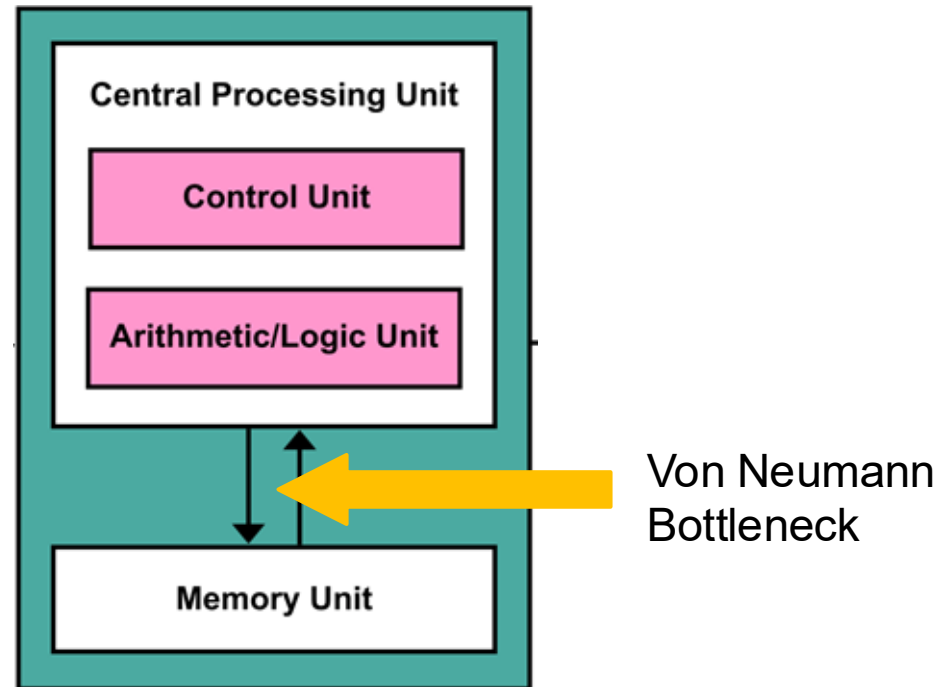


Von Neumann Architecture

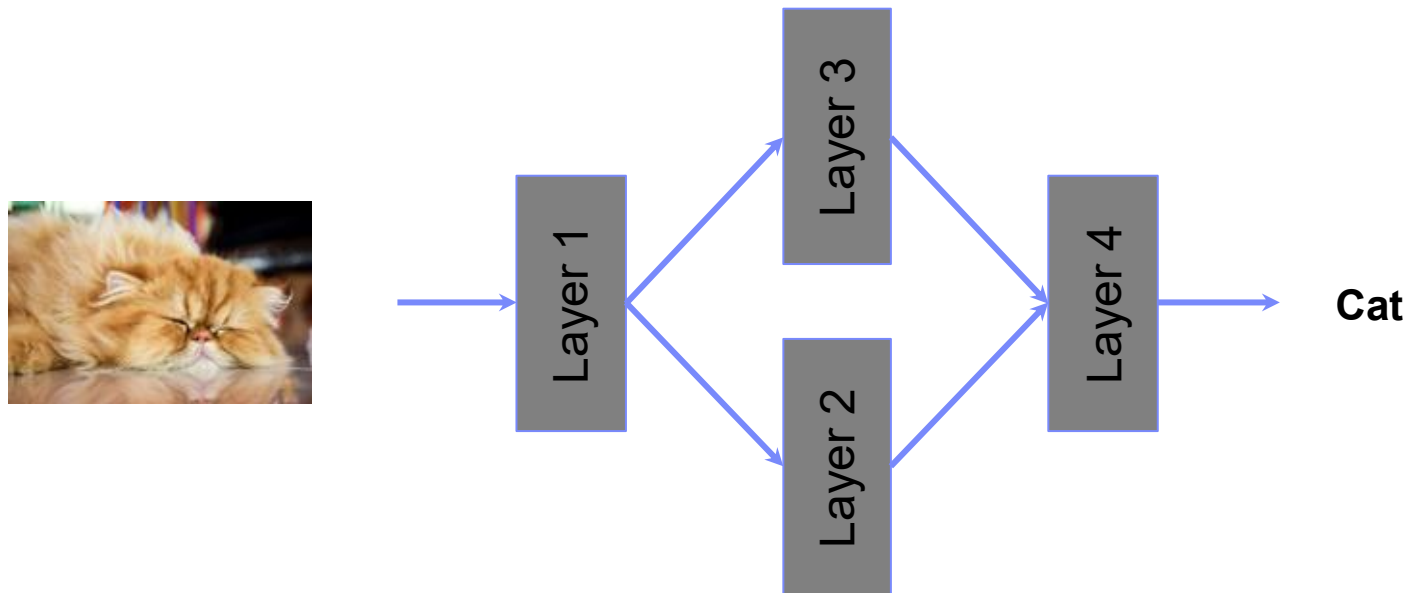


How do we map a DNN onto a computer chip?

- Program? Instructions? Data?



Computational View of a DNN



Computational View of a DNN

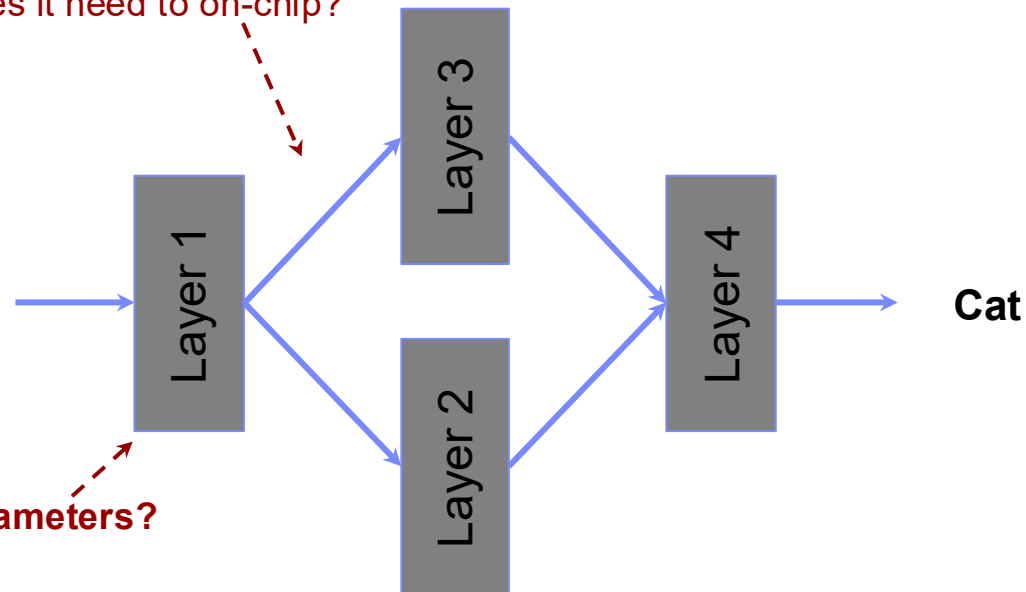
1. Memory

Size of activation?

- Does it fit on-chip?
- How long does it need to on-chip?



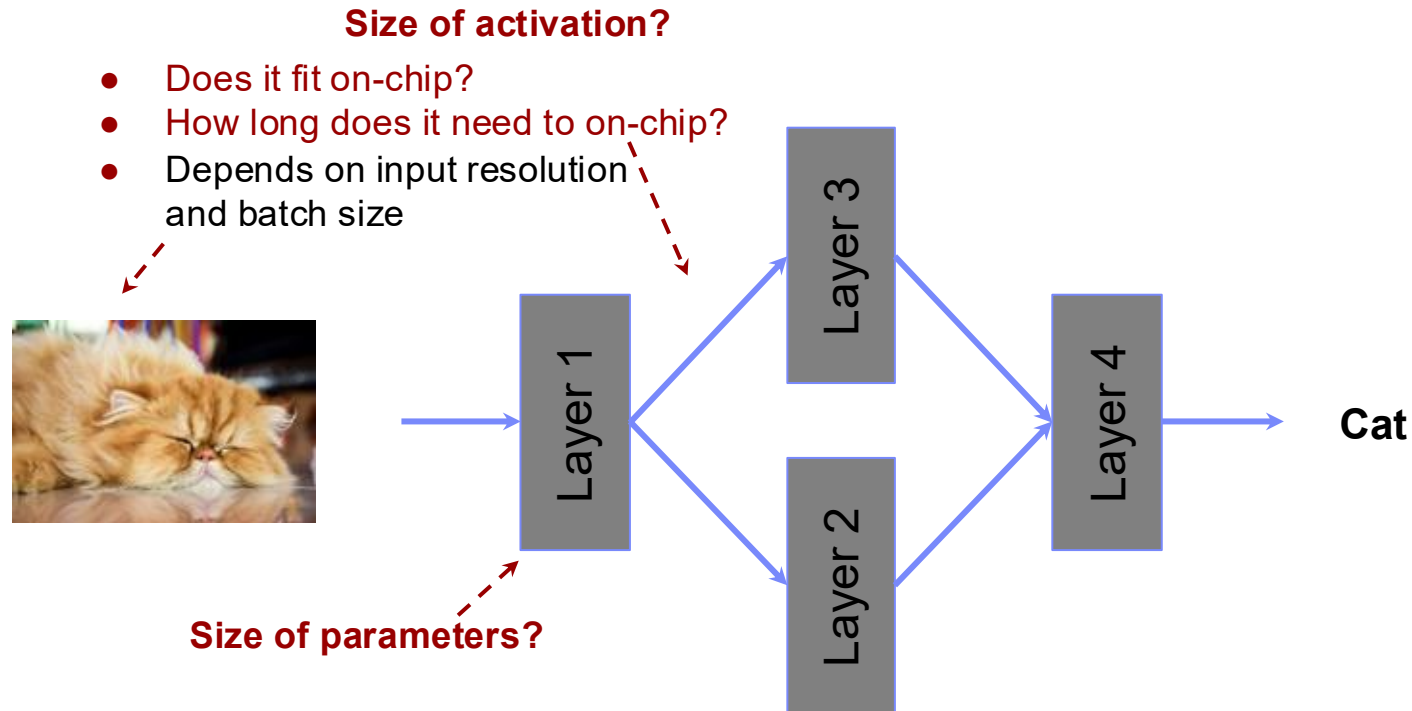
Size of parameters?



- Does it fit on-chip?
- How long does it take to load from off-chip?
- Can I overlap loading with computation?
- Is there reuse of the loaded parameters?

Computational View of a DNN

1. Memory



- Does it fit on-chip?
- How long does it take to load from off-chip?
- Can I overlap loading with computation?
- Is there reuse of the loaded parameters?

Computational View of a DNN

1. Memory

2. Compute

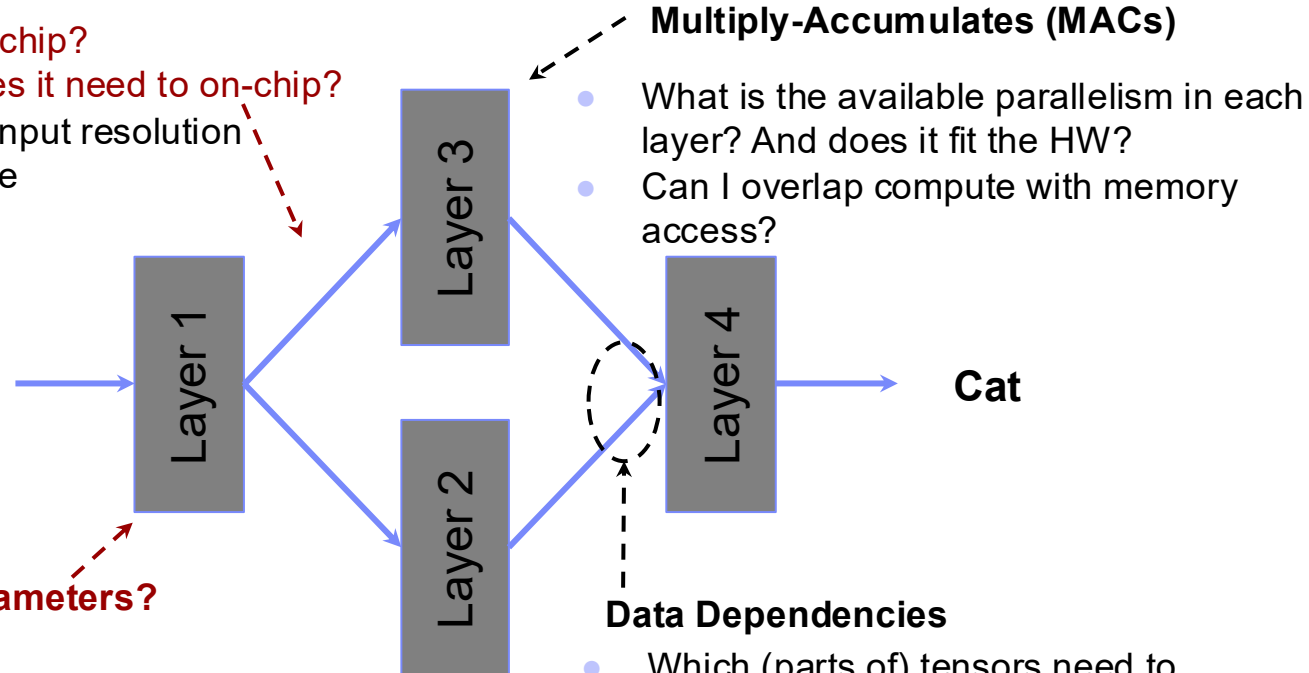
- Does it fit on-chip?
- How long does it need to on-chip?
- Depends on input resolution and batch size



Size of activation?

Size of parameters?

- Does it fit on-chip?
- How long does it take to load from off-chip?
- Can I overlap loading with computation?
- Is there reuse of the loaded parameters?



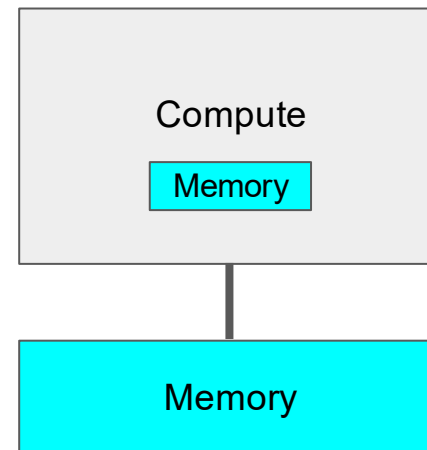
Memory-bound vs. Compute-bound

❧ Does it take more time to:

- Do the computation?
- Fetch data from memory?

❧ Depends on a number of factors:

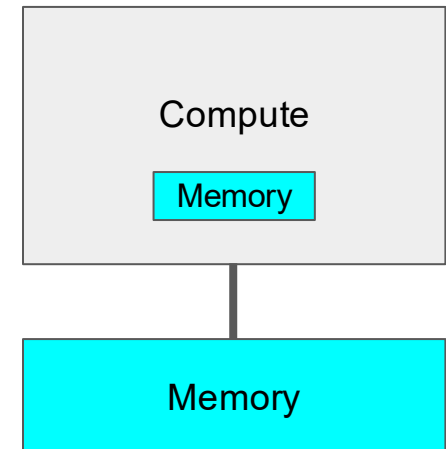
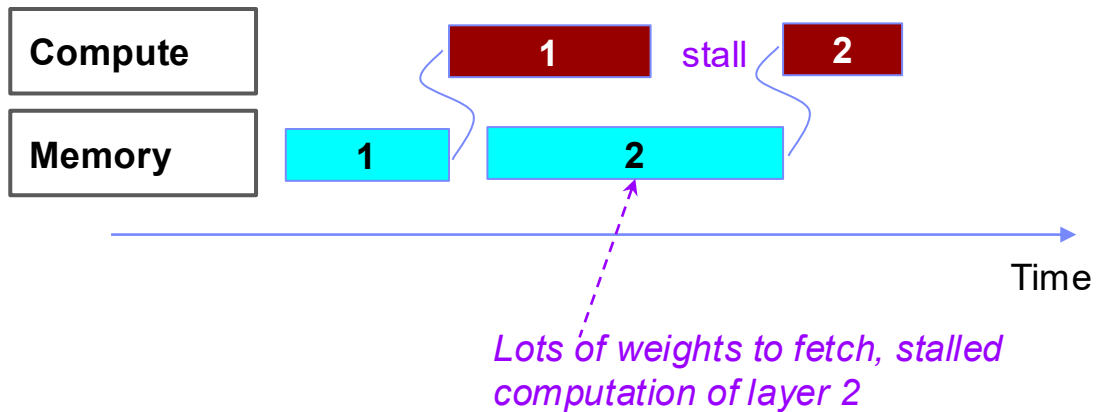
- Memory bandwidth
- Compute speed/parallelism
- Size of operands
- Size of (intermediate) result
- On-chip buffering resources
- ...



We will focus on DNN-related factors

Overlap Memory with Computation

- Fetch parameters from memory
- Compute layer using fetched params



Overlap Memory with Computation

- Fetch parameters from memory
- Compute layer using fetched params

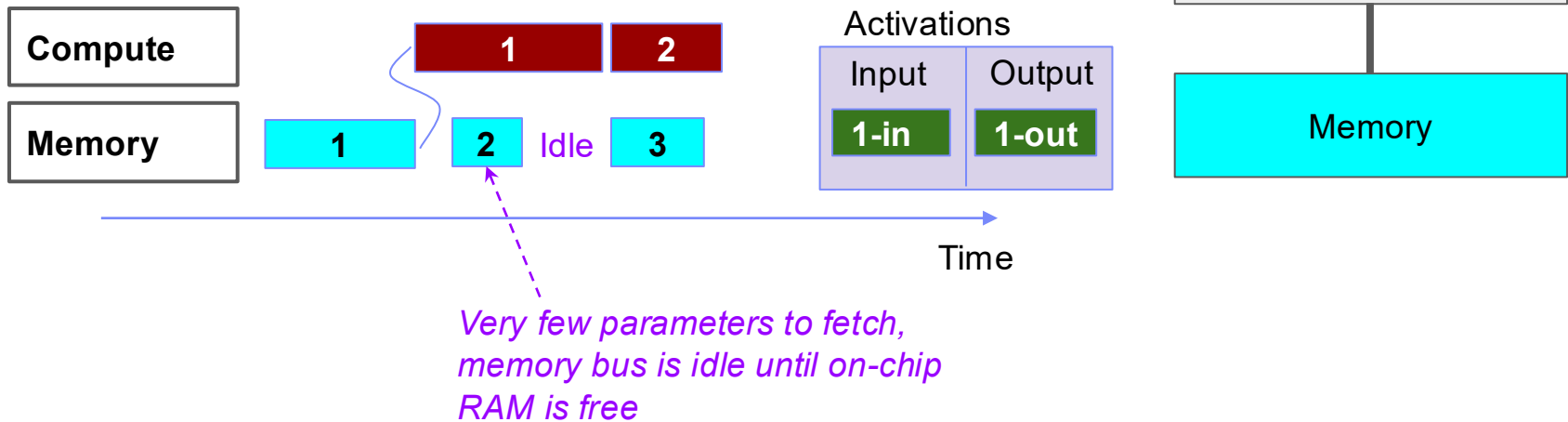
Double-buffering

Parameters

Current	Next
1	2

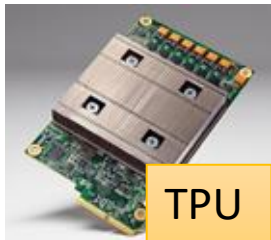
Activations

Input	Output
1-in	1-out



Diversity of Hardware

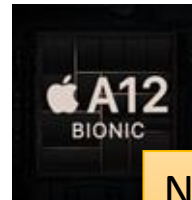
- ❌ Good to have a mental model of the hardware architecture, but we actually need to run machine learning on a *very wide variety* of hardware
- ❌ At the first order, the differences are in the compute and memory capabilities



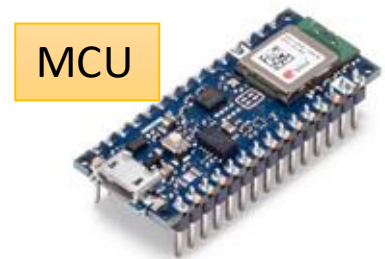
TPU



FPGA



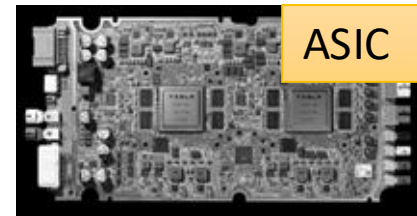
NPU



MCU

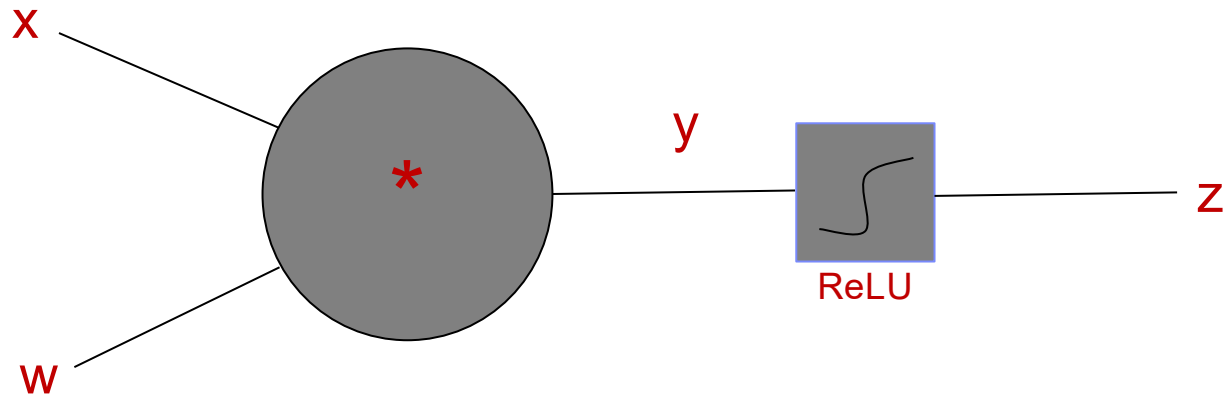


GPU



ASIC

Single “Neuron”

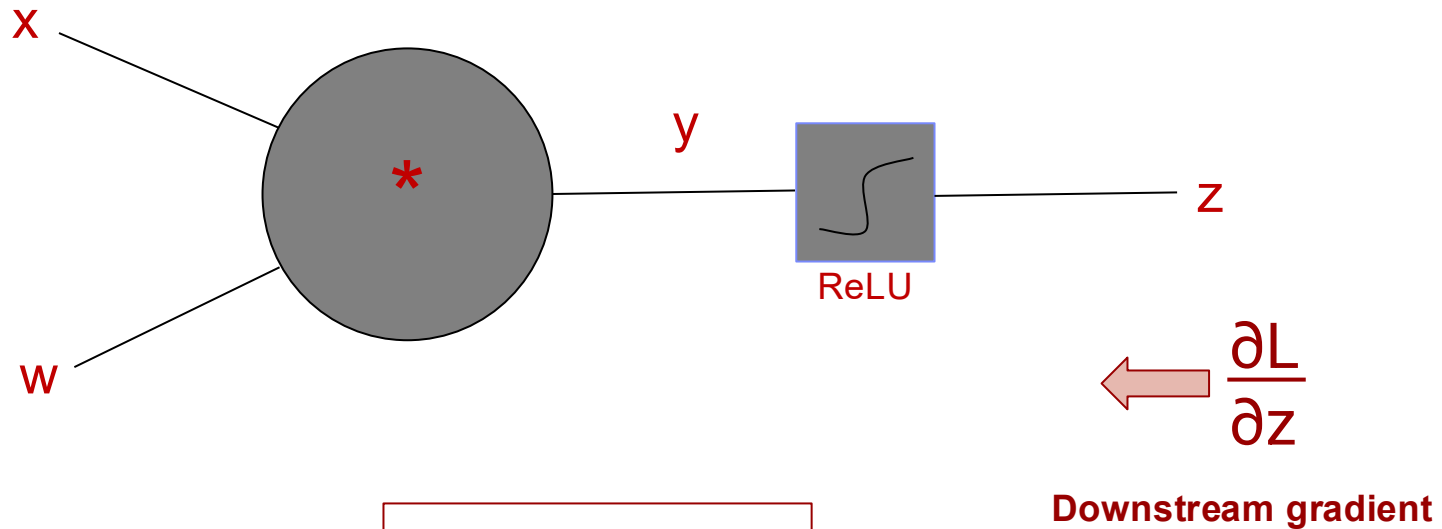


Forward pass:

$$z = \text{ReLU}(x * w)$$

1 multiplication

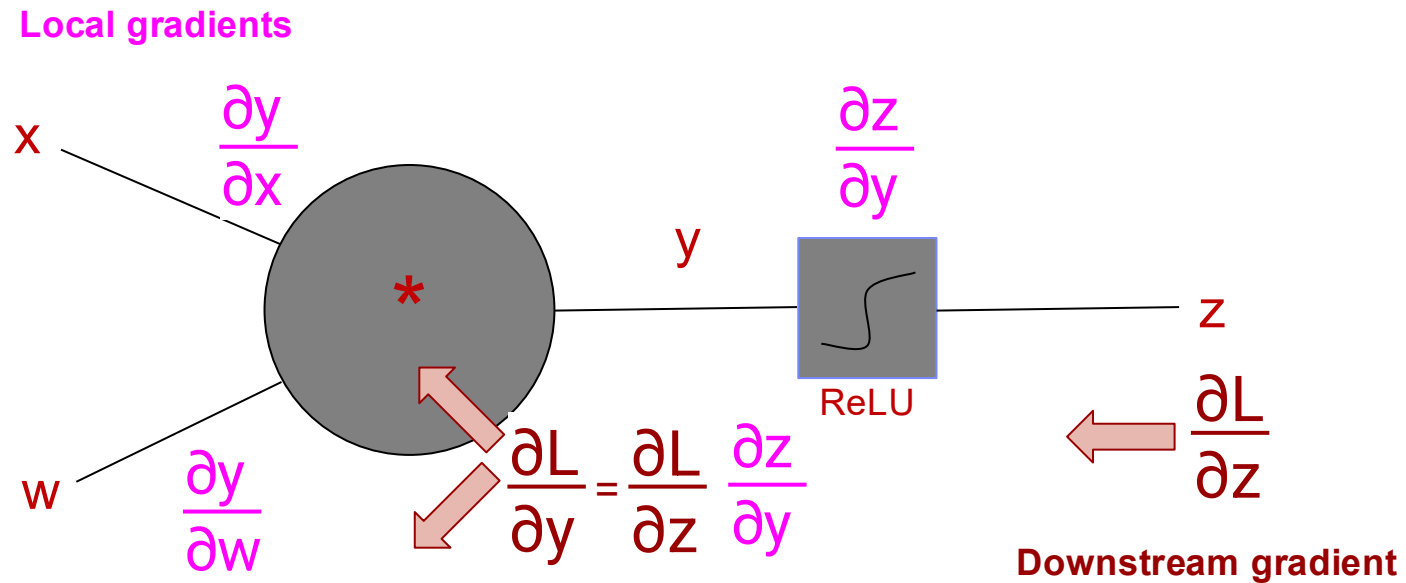
Single “Neuron”



Backward pass:

$$\frac{\partial L}{\partial w} =$$

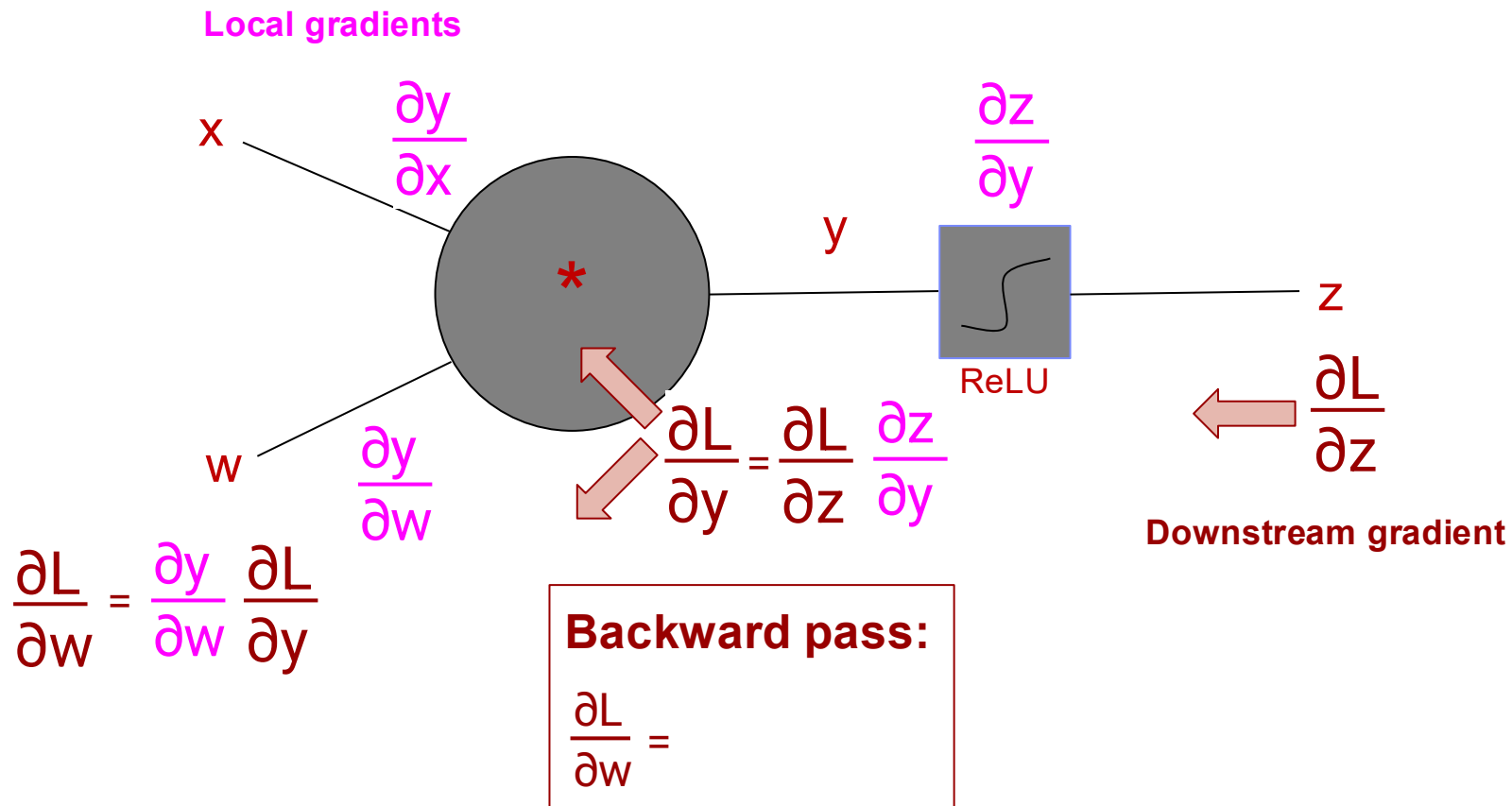
Single “Neuron”



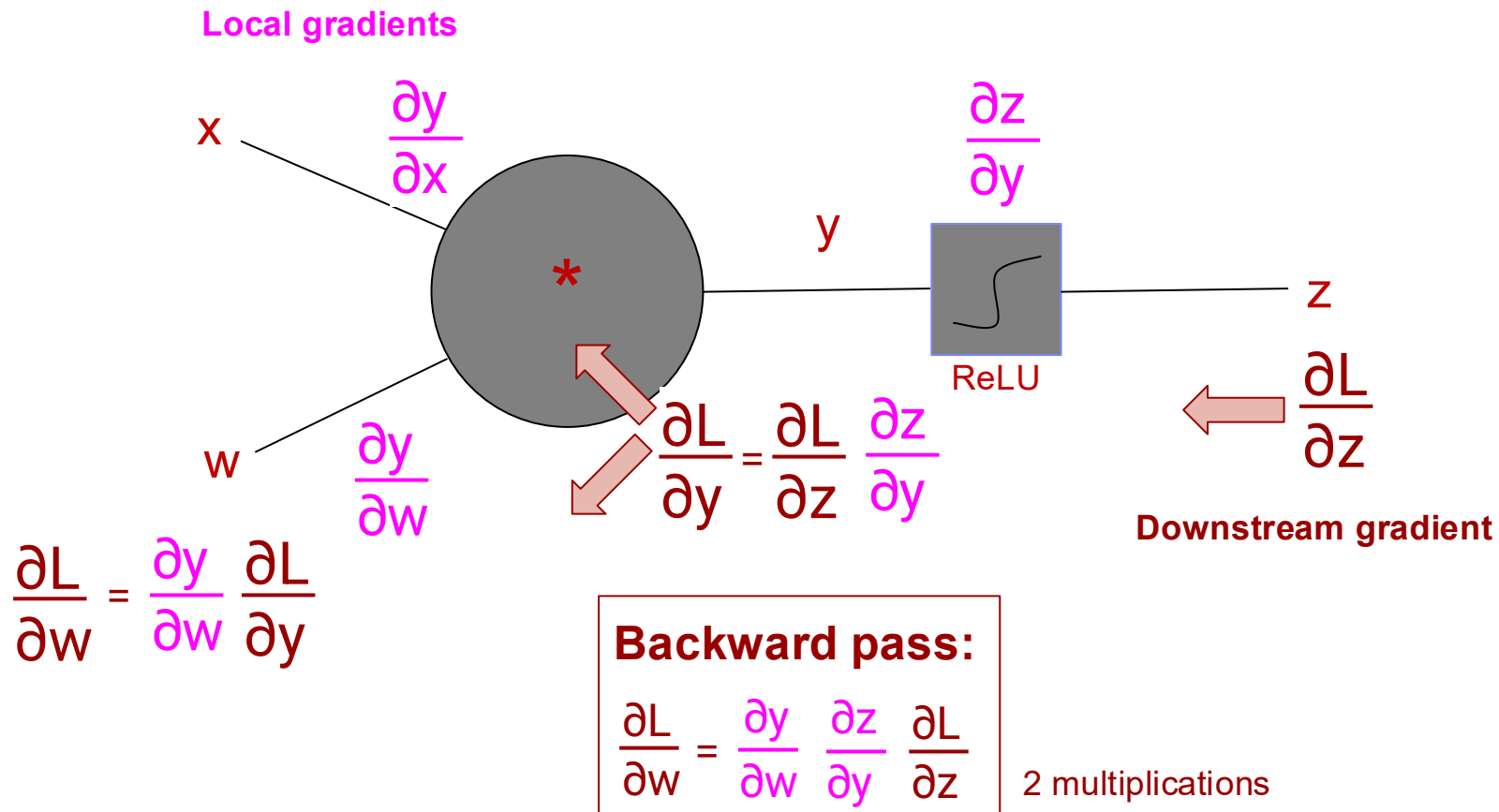
Backward pass:

$$\frac{\partial L}{\partial w} =$$

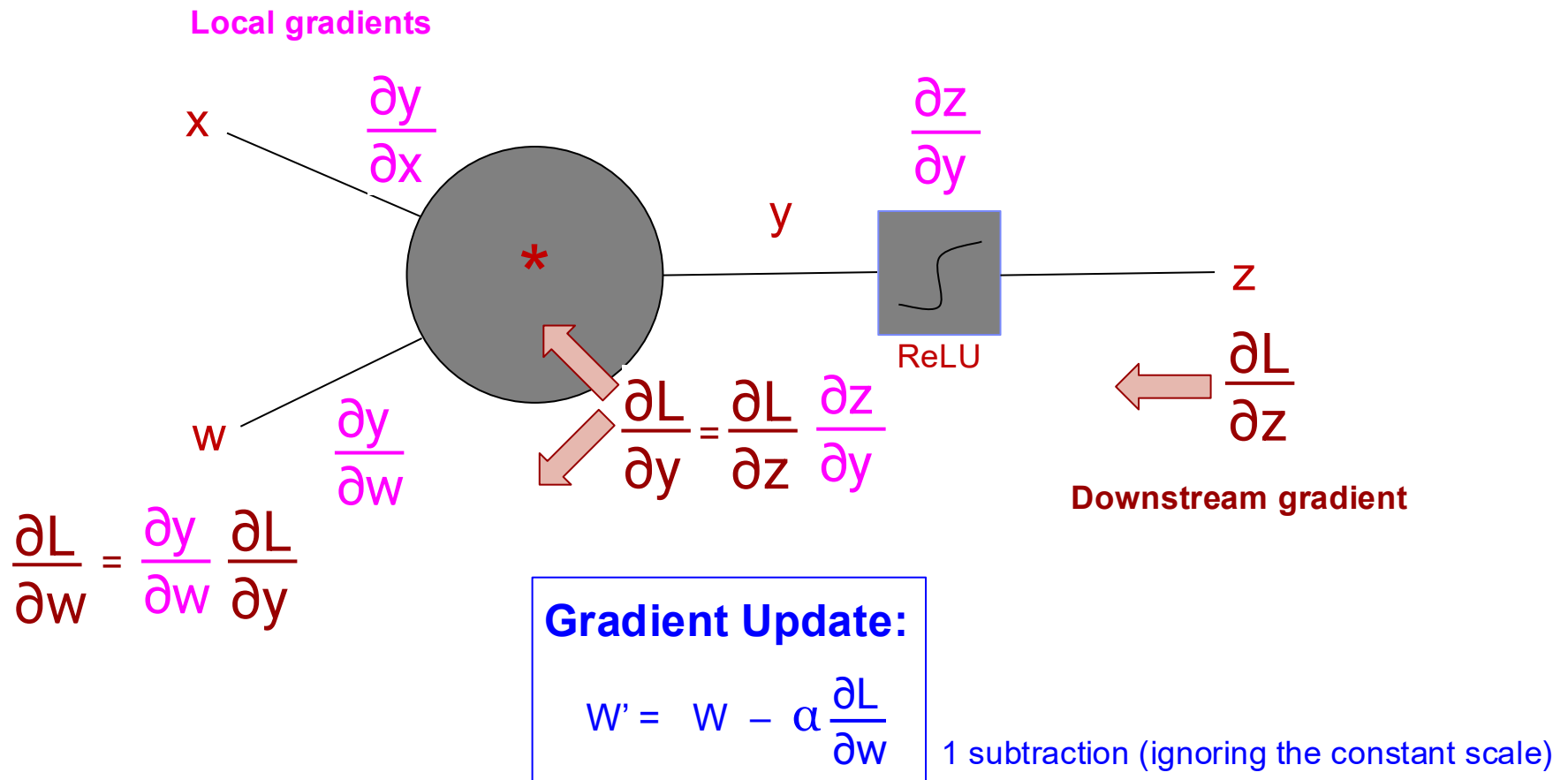
Single “Neuron”



Single “Neuron”

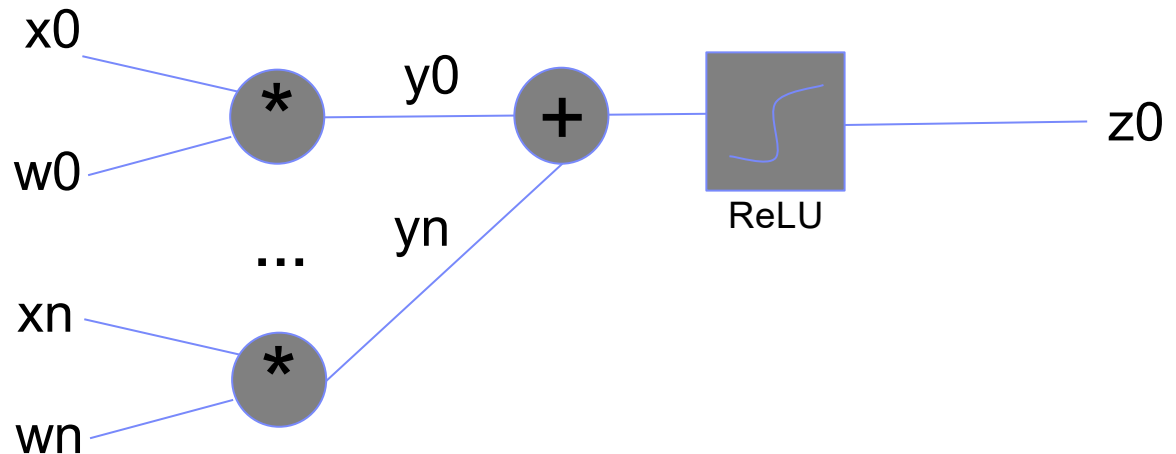
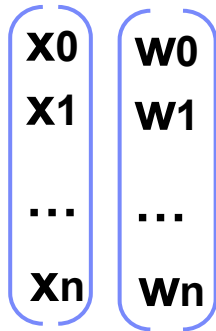


Single “Neuron”



Single “Neuron”

$$y = x \cdot w$$



Part of a:

Forward pass:

$$z = \text{ReLU}(x \cdot w)$$

Backward pass:

$$\frac{\partial L}{\partial w} = \frac{\partial y}{\partial w} \frac{\partial z}{\partial y} \frac{\partial L}{\partial z}$$

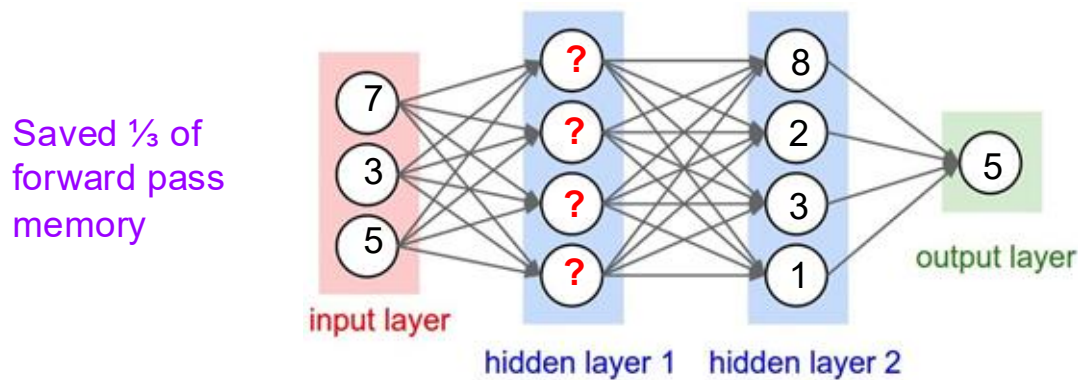
Gradient Update:

$$w' = w - \alpha \frac{\partial L}{\partial w}$$

Ratio of operations in backward:forward pass $\sim 2:1$

DNN Model Checkpointing

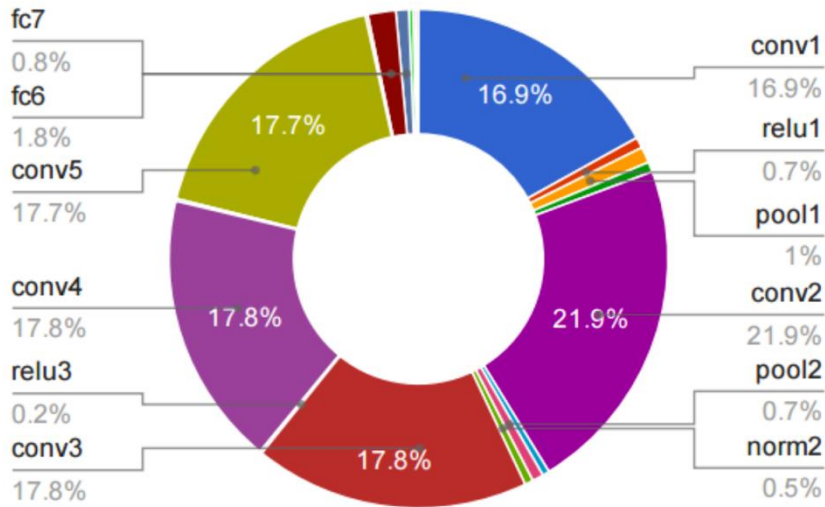
- ❧ Generally, need to store all activations from forward pass, to use during backward pass
- ❧ Checkpointing skips some of those activations and recalculates them on the fly during the backward pass
- ❧ More computations, less memory!



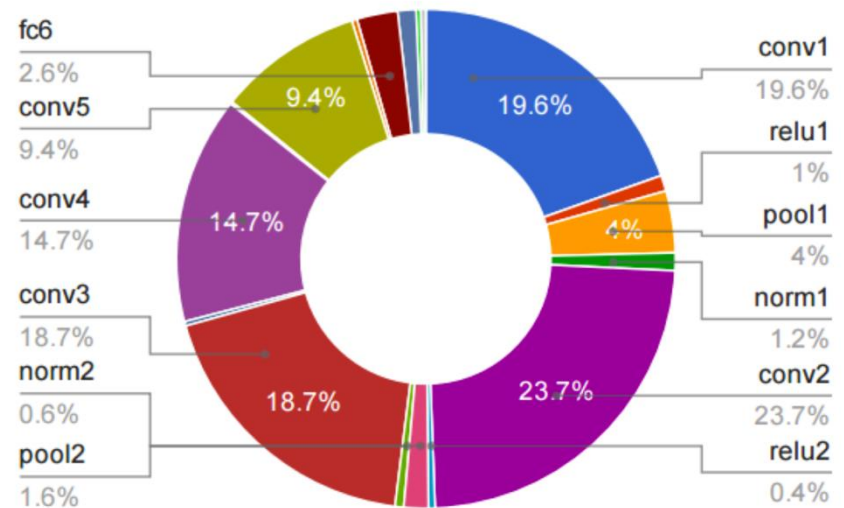
Breakdown of DNN Computations

Most of the time is spent in: (1) Convolutions, (2) Linear (or matrixmul)

GPU Forward Time Distribution



CPU Forward Time Distribution



Source: Y. Jia